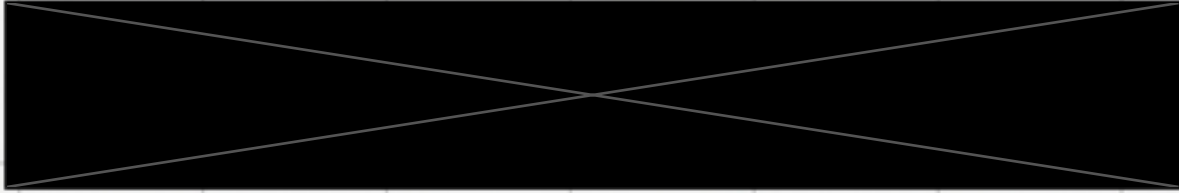
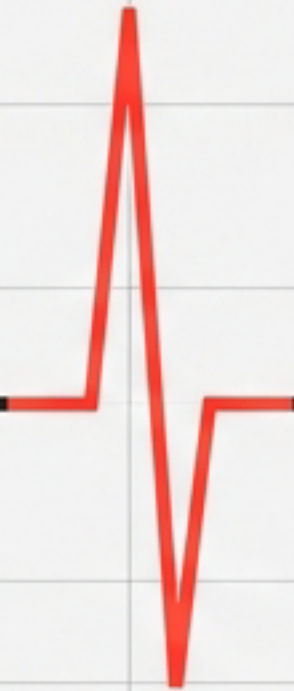


The Bellwether Test



Why “Time-to-Replication”
is the ultimate measure of
codebase health.

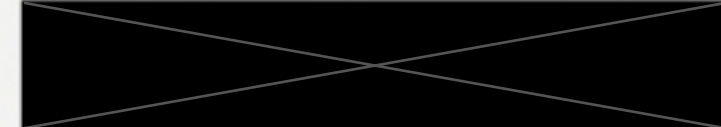


When a bug is found, how quickly
can you replicate it as a test?

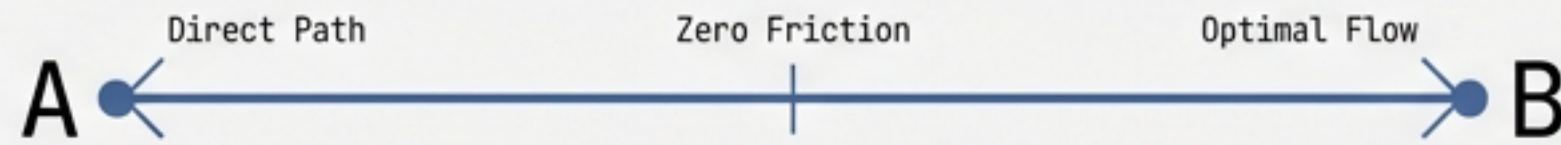
NOTE: METRIC = REPLICATION SPEED,
NOT RESOLUTION SPEED.

THE PULSE OF COMPETENCE

NOT FIX IT.

 REPLICATE IT.

FAST REPLICATION



SLOW REPLICATION



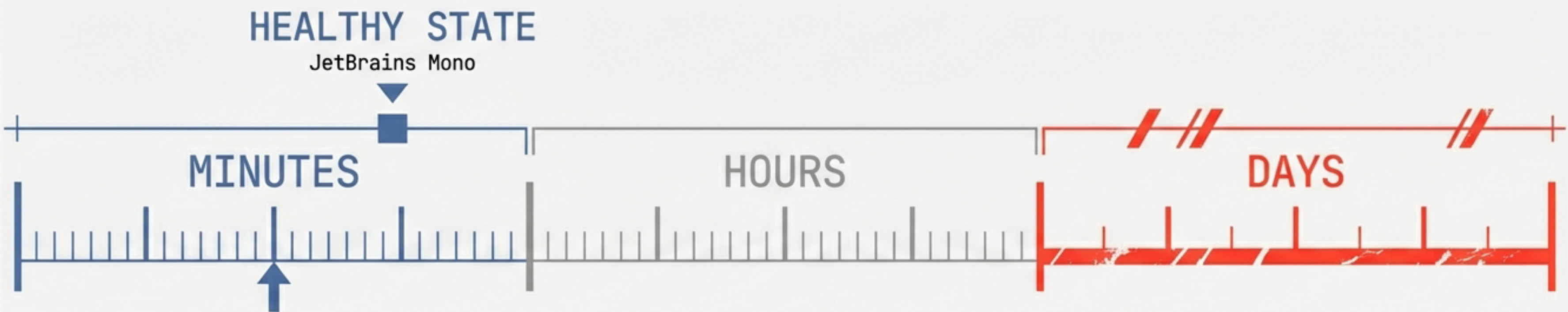
GOAL:

Write a test that reproduces the exact behaviour—the browser navigation, the error message, the feature failure.

THE BINARY SIGNAL:

- ✓ **FAST:** Good Infrastructure (Plumbing works).
Good Understanding (No opacity).
- ✗ **SLOW:** Inadequate tooling.  Technical Alerts
Team ignorance of system.

THE METRIC: TIME TO FIRST REPLICATION.

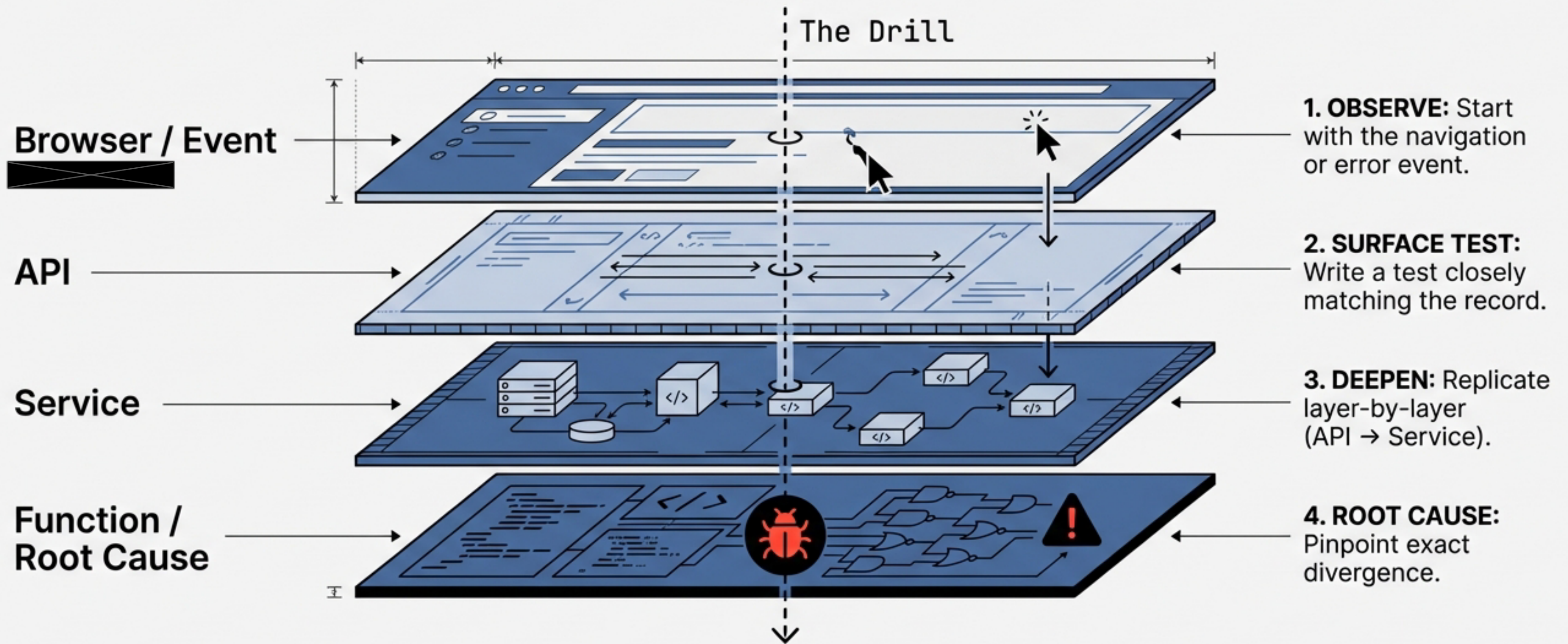


DEFINITION	Duration from Bug Observed to Reproduction Test Exists.
DEPTH	Surface (Browser) vs. Root Cause (Function).
WHO	Original Developer vs. Any Team Member.

A team where anyone can replicate a bug in **minutes** has a fundamentally different quality posture.

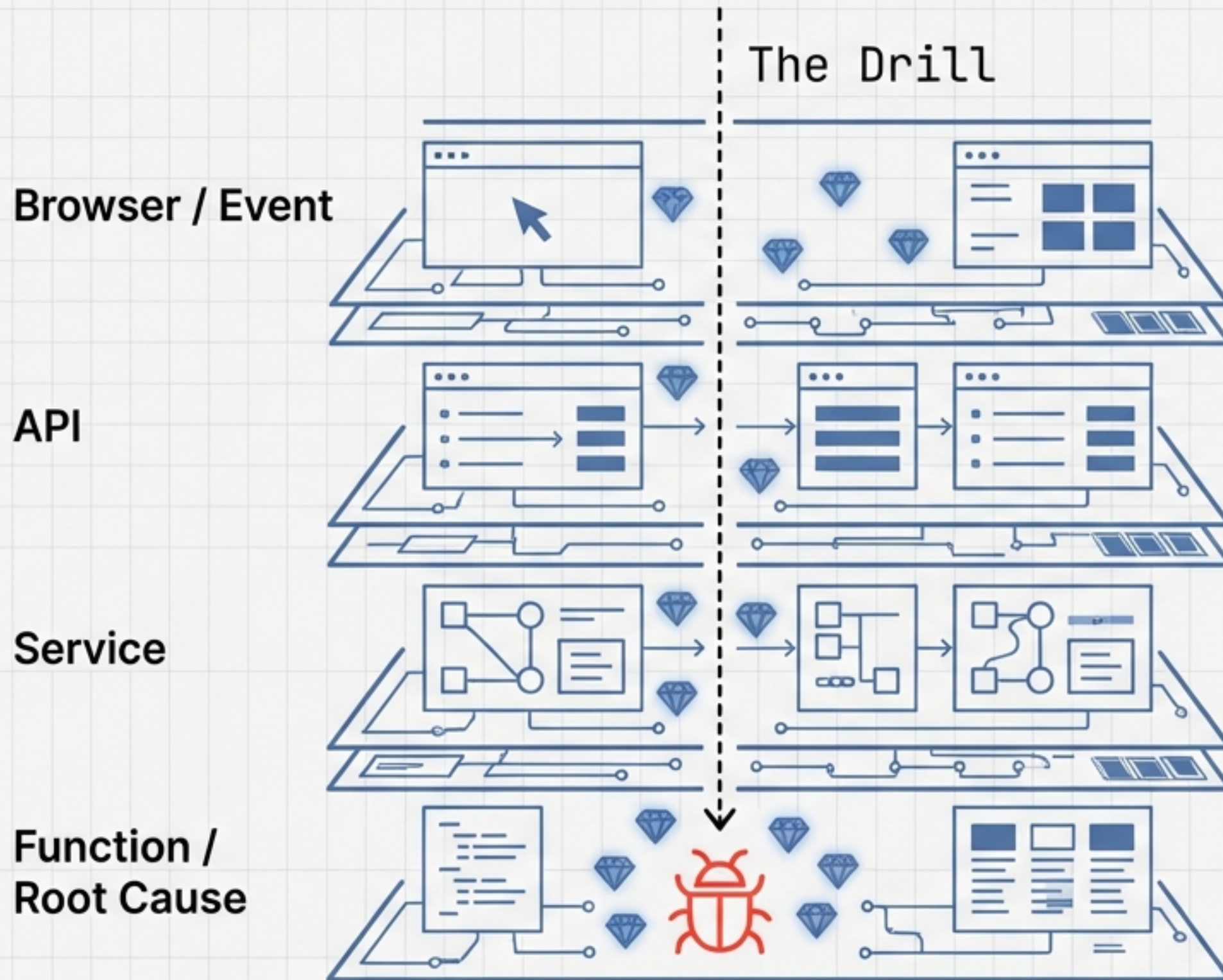


THE METHODOLOGY: BUG WALKING



Don't jump to the fix. Walk the path.

A TRAIL OF PROTECTION



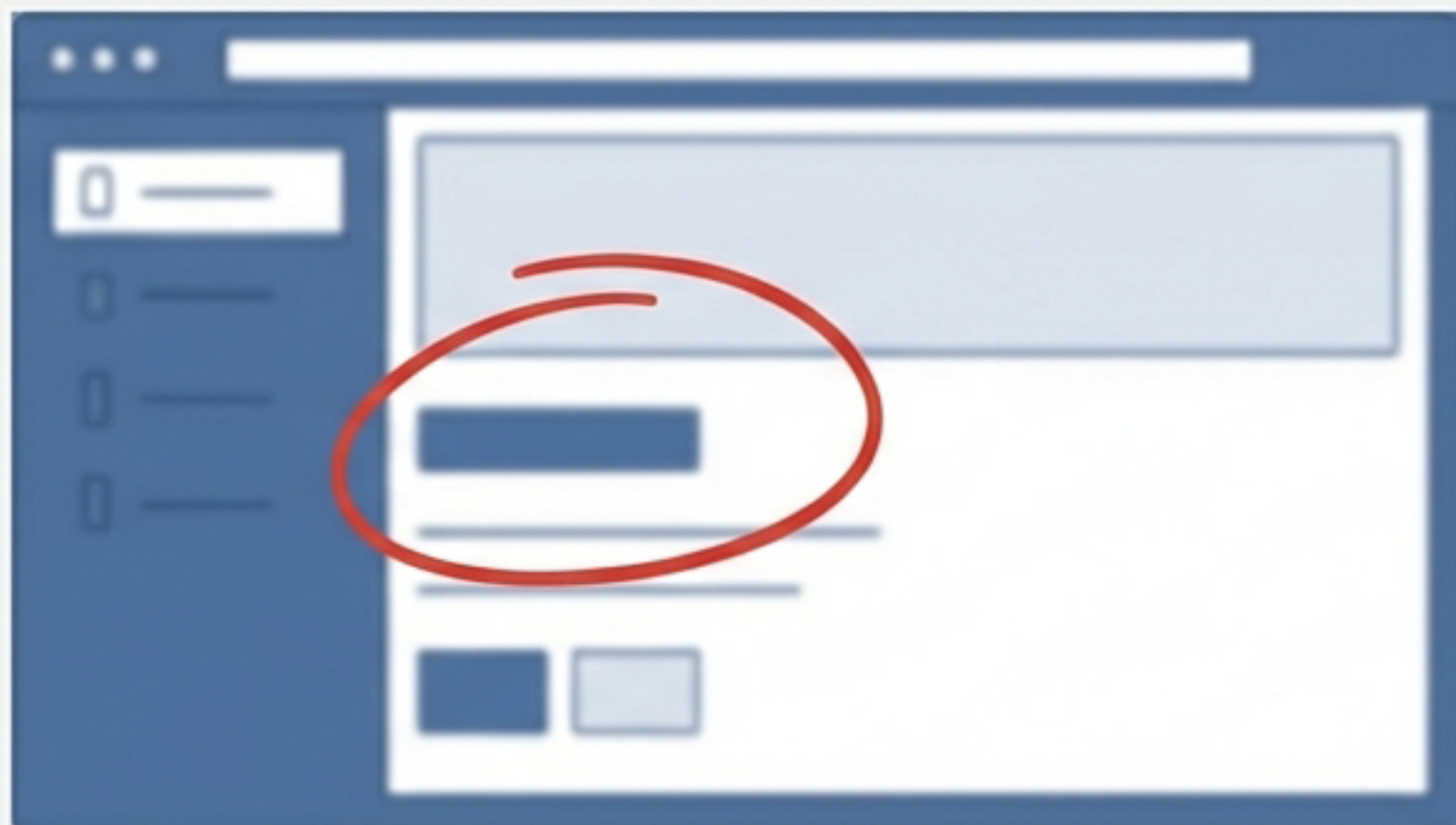
At every layer of the “walk,” you are writing tests. By the time the bug is found, you have a documented path from symptom to cause.

RESULT: These tests remain in the codebase as permanent regression protection.



MANAGERS MUST COMMUNICATE IN CODE

THE OLD WAY



Ambiguous. Requires investigation.
“Hey, it's broken.”

THE NEW WAY



```
import { test } from '@playwright/test';

test('bug_reproduction_123: should fail when clicking submit
with empty field', async ({ page }) => {
  // Navigate to the specific page
  await page.goto('/submission-form');

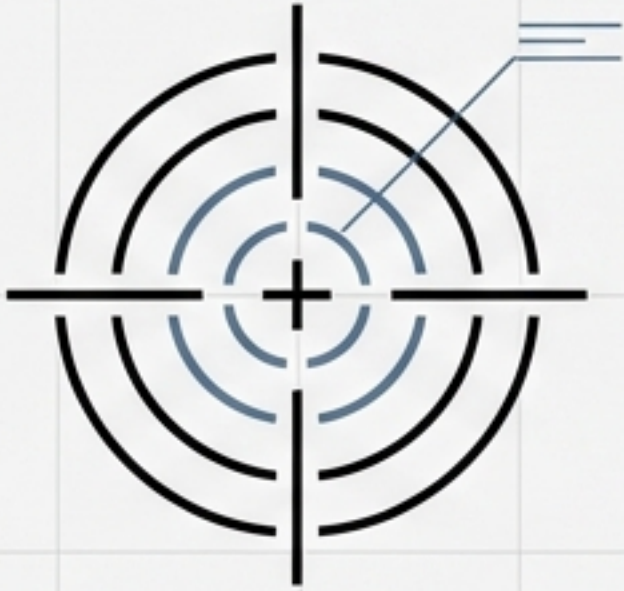
  // Leave required field empty and click submit
  await page.locator('input#required-field').fill('');
  await page.locator('button#submit-btn').click();

  // Assert the expected failure or error message
  await expect(page.locator('.error-message')).toBeVisible();
  await expect(page.locator('.error-message')).toHaveText(
    'Field cannot be empty.');
});
```

Precise. Executable.
“Here is the behaviour.”

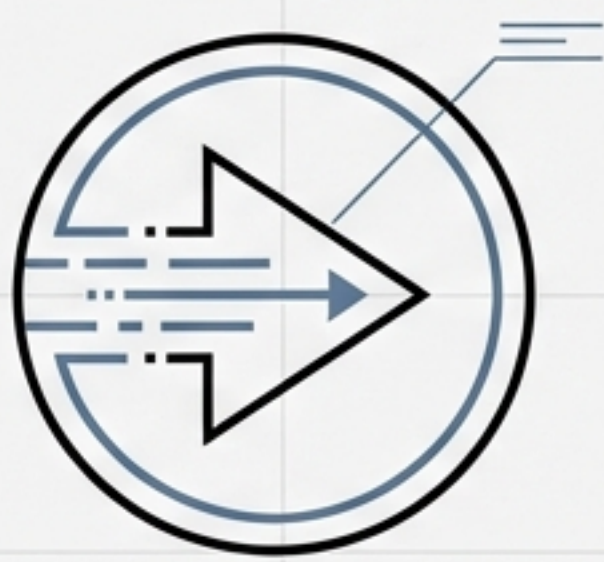
The Expectation: Managers or Senior Engineers should write
the replication test (or use an LLM to generate it).

WHY CODE IS BETTER THAN ENGLISH



PRECISE

No ambiguity about the observation.



EXECUTABLE

Run it. See the bug immediately.



VERIFIABLE

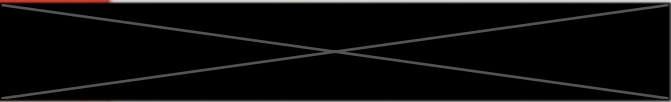
When fixed, the test passes.



DOCUMENTED

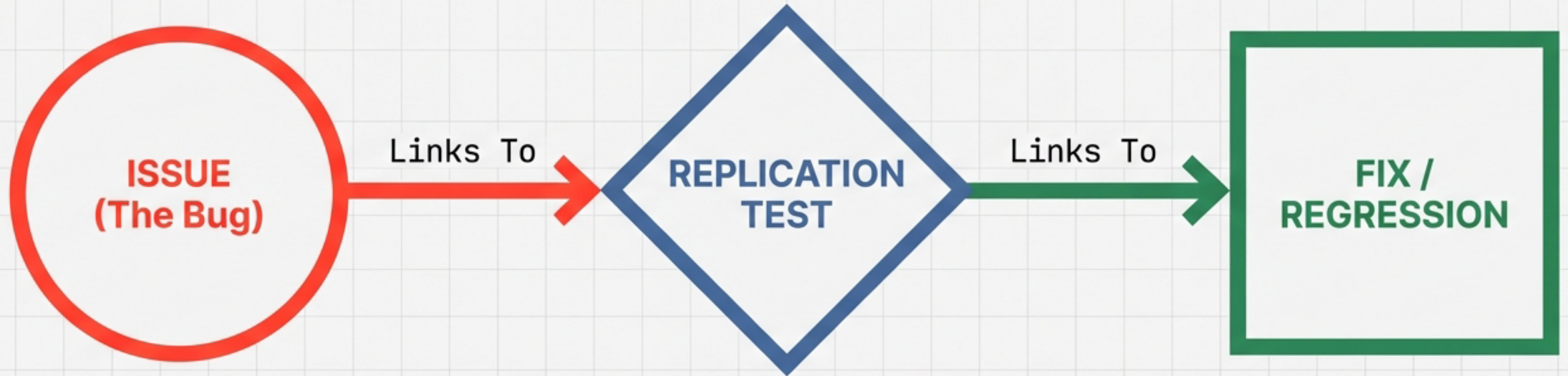
A permanent record of what 'fixed' means.

SIGNAL: If a manager cannot write the test, the codebase is too opaque or the infrastructure is too weak.



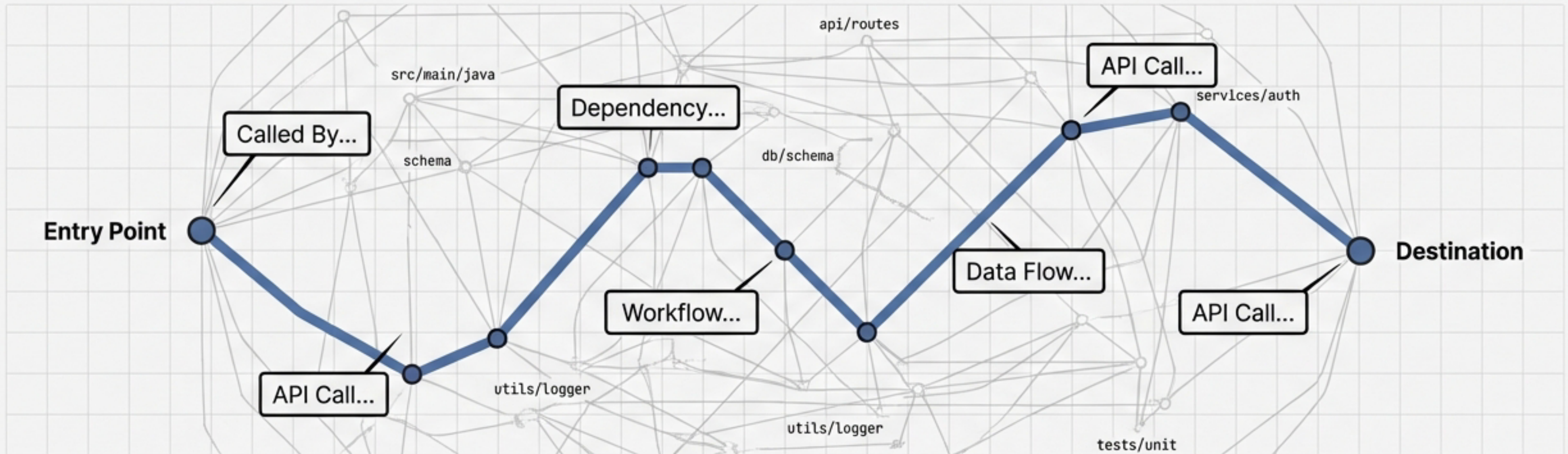
Enabling Technology: Issues FS

Operationalising the workflow through a connected graph.



Bug-to-Test Traceability: The entire trail is a connected graph, traceable from any point.

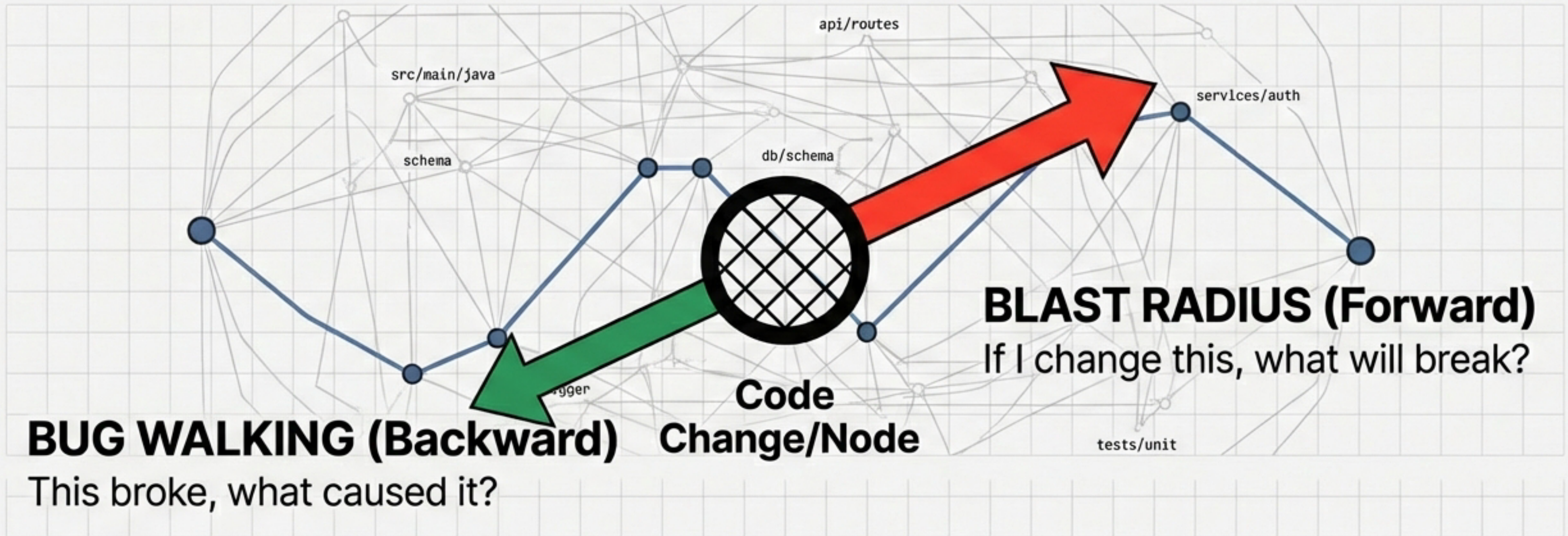
Walking with the Sidecar



Problem: Walking by memory is error-prone.

Solution: The Sidecar provides the code graph. It tells you, "This function is called by these paths." The walk is guided by the graph.

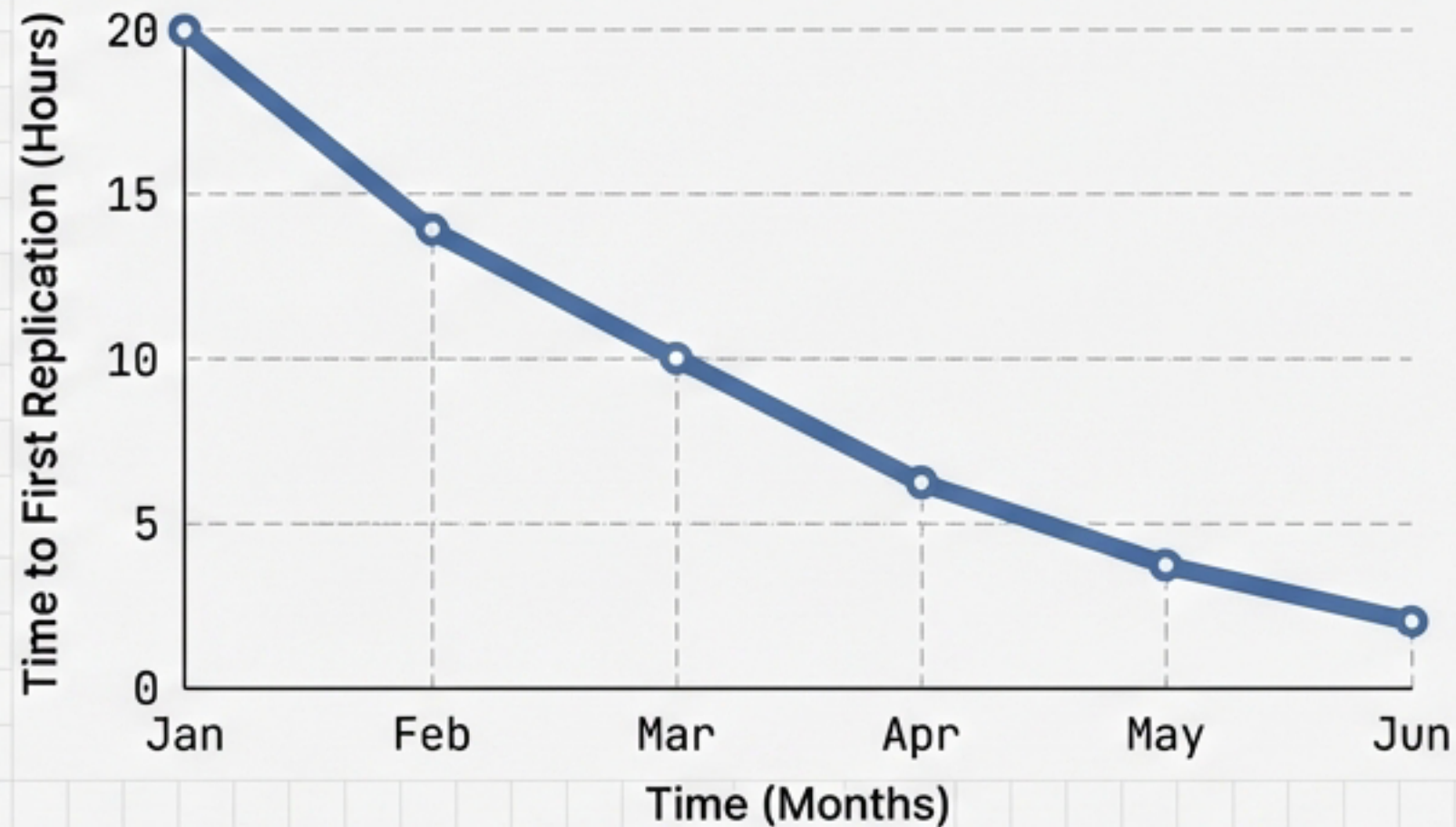
Inverting the Blast Radius



Mechanism: Both are traversals on the same Code/Issue graph. Same infrastructure, opposite direction.

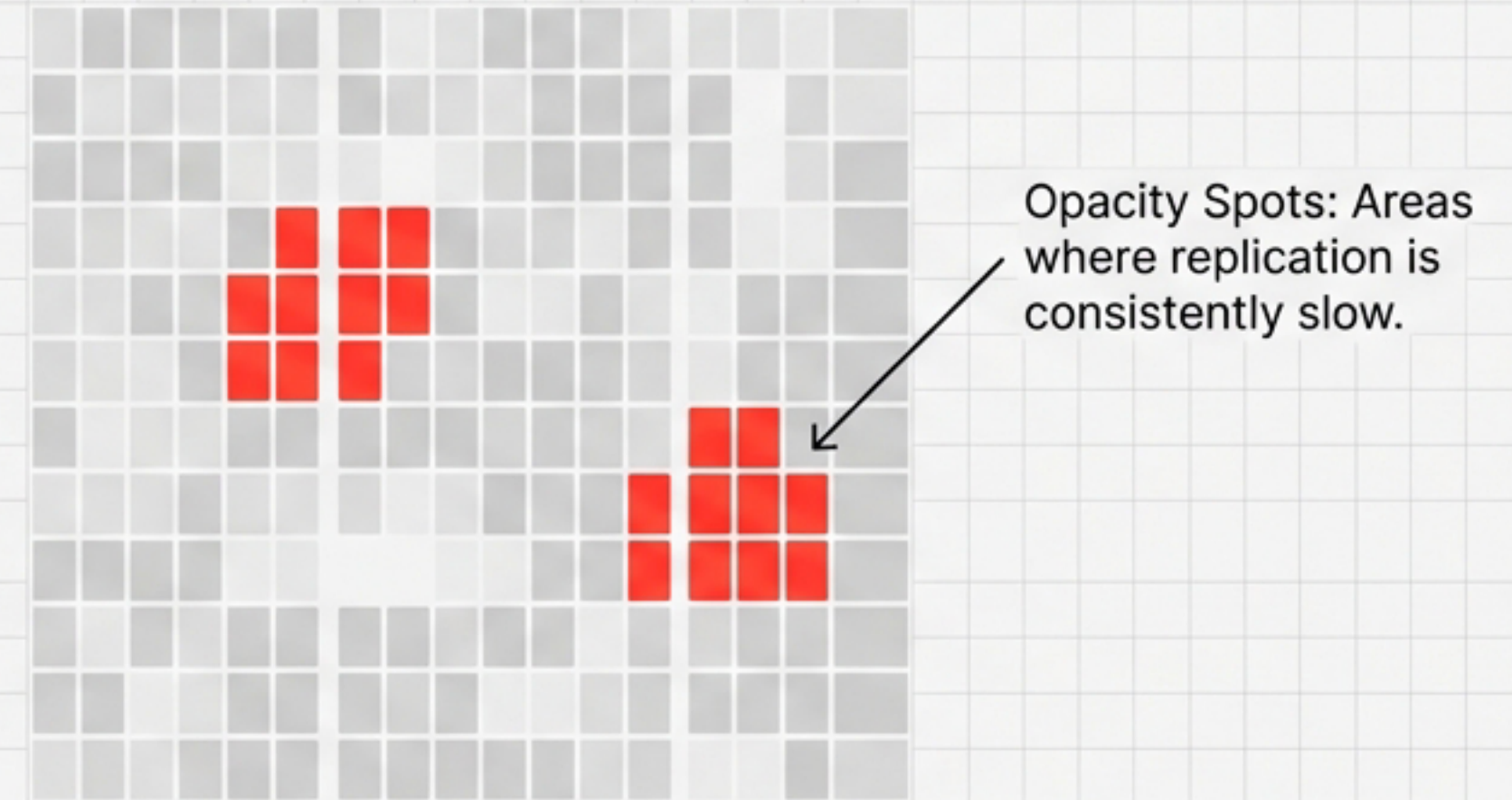
Bellwether as a Dashboard Metric

Time to First Replication Trend



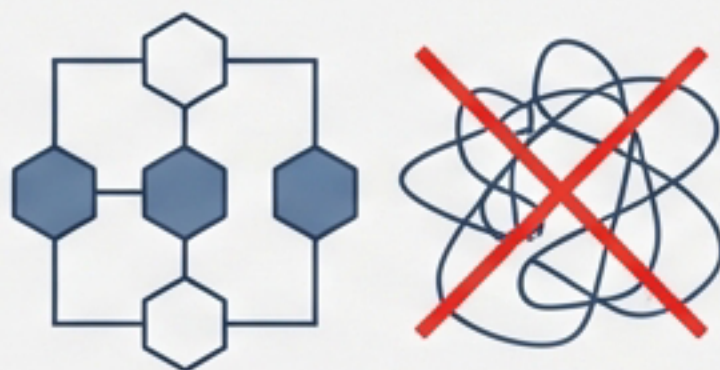
Tracking 'Time to First Replication' allows us to identify and fix organisational blind spots.

Codebase Heatmap: Opacity Spots



Mandates for the Agents

Architect

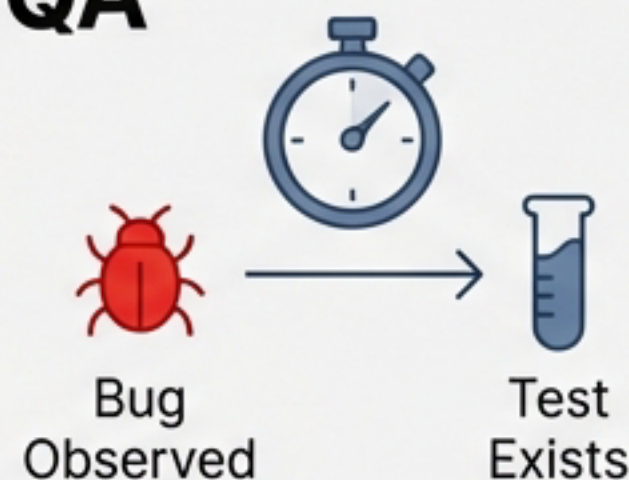


Ensure state setup is **easy**.

No dependency hell.

```
state.setup()  
no_dependency_hell
```

QA



Measure the metric.

Track "Bug Observed" → "Test Exists" time.

```
time(BugObserved → TestExists)
```

Dev

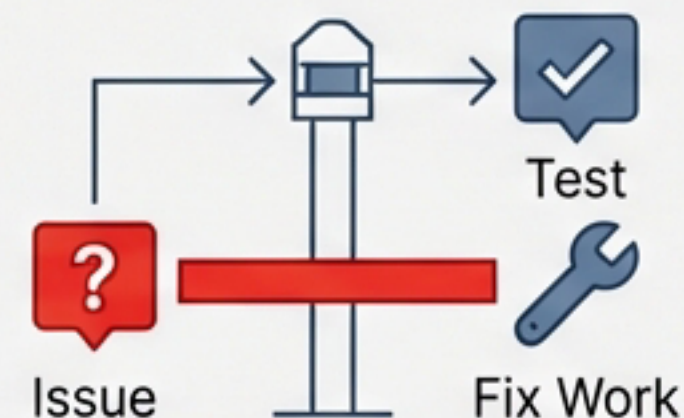


Replicate THEN fix.

Every fix must follow the Bug-Walking pattern.

```
replicate()  
fix()  
BugWalking.follow()
```

Conductor

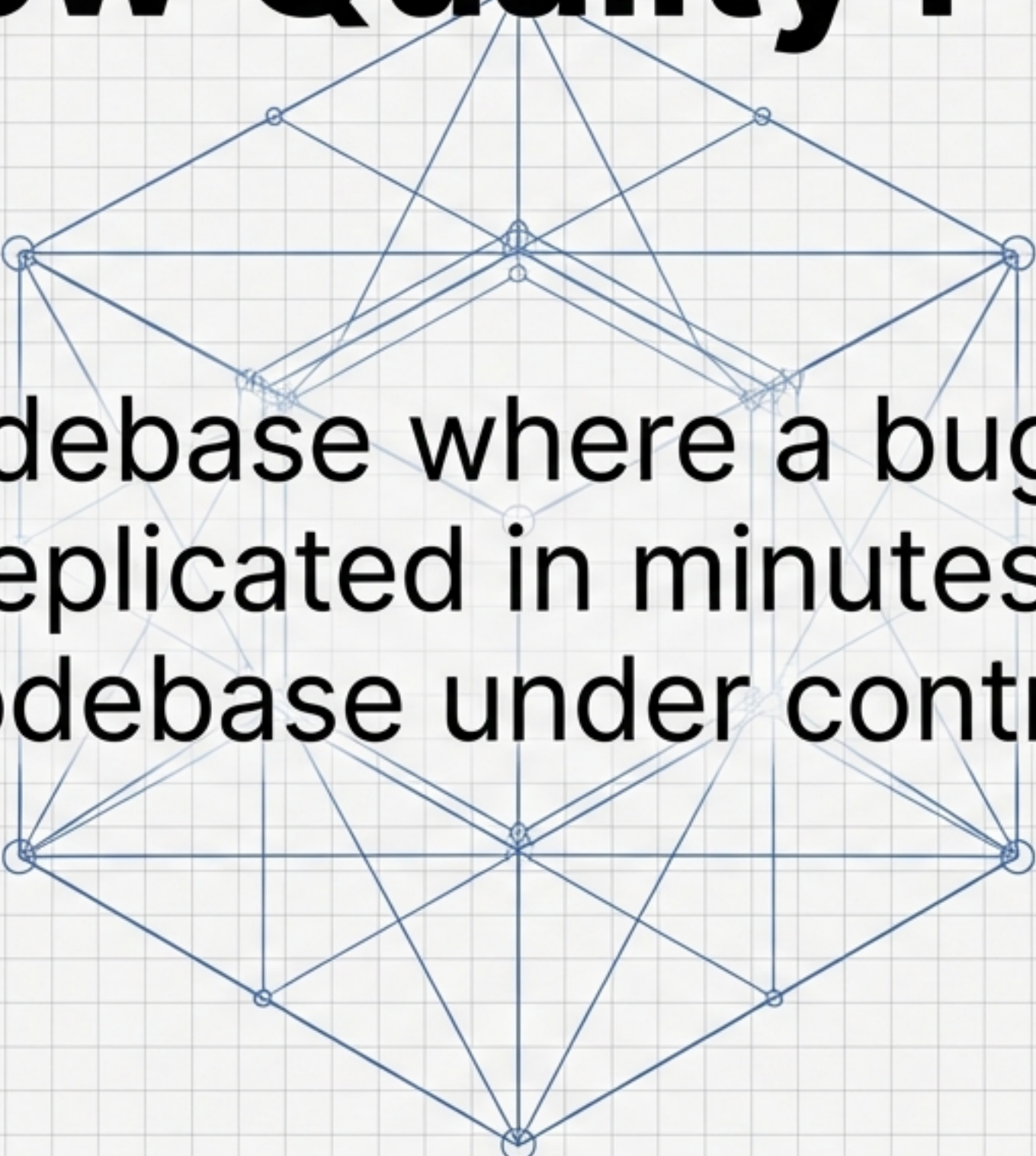


Gatekeeper.

Ensure issues link to replication tests before fix work begins.

```
Issue.link(ReplicationTest)
```


The New Quality Posture



A codebase where a bug can
be replicated in minutes is a
codebase under control.

Stop fixing in the dark. Start replicating in the light.